

Bitcoin Price Prediction: Report 2 - Modelling Results

1 Introduction & Task Selection

This report outlines the methodology chosen to tackle a Bitcoin (BTC) Price Prediction Regression task. The two datasets created in the previous report, one capturing long-term BTC close values and the other a smaller dataset incorporating Twitter sentiment, were employed for this task. A variety of models were investigated. Due to the ever increasing usage of powerful neural approaches, it was decided to compare neural and non-neural approaches with respect to performance but also environmental costs. It could be seen that a univariate LSTM model performed best with a MAE, RMSE and sMAPE of 0.04, 0.05, 5.57% respectively for Dataset 1 and 0.01, 0.01 and 15.27% respectively for Dataset 2. However, it also saw three times the amount of energy consumption and more emissions than its non-neural counterparts. While, its performance was far greater, this report encourages more work into efficiently creating and finetuning such models. The rise of neural approaches and LLMs has led to an influx of usage of its applications in all domains. While providing exceptional results in many areas, it is worth considering non neural alternatives, especially with respect to computational resources and energy consumption. Comparing neural with non-neural approaches allows us to understand the performance - environmental trade off when deciding on the approach for a given problem. Therefore in this project a number of neural and non neural approaches will be investigated and in order to compare environmental costs, code carbon will be employed in all colab notebooks allowing us to track and compare costs. For further investigation, all notebooks, models and emission results have been included in the same folder as assignment 1. As these notebooks heavily rely on utilizing google drive, file paths must be given new paths to run successfully.

2 Literature Review

2.1 ARIMA, ARIMAX & Linear Regression

For interpretability, lower computational resources and environmental considerations, non-neural approaches were investigated. It was shown in [1] that both Singular and Multiple Linear Regression could be employed to predict Bitcoin price. In this instance, volume was used in Multiple Linear Regression. The result indicated that adding more history to the neural and non-neural approaches helped, i.e., lag features should be

employed. No computational/environmental expectations were compared, leaving a gap to be further explored. Therefore, in this experiment, lag features will be added and the multiple linear regression will investigate multi-modal features, such as Twitter sentiment.

Work by Zhu in [2] compared ARIMA and SARIMA with Multiple Linear Regression, indicating that all results proved promising for all three approaches. Here, it was stated that all predictors must be linearly independent for simple Multiple Linear Regression. Linear Regression is by far the most simple approach to Bitcoin Price Prediction, assuming that the Close Value is just some linear combination of itself or other factors. Further work could investigate regularising the model to provide a more robust Linear Regression model capable of dealing with more complex data sources.

Other work [3], [4] also exploring ARIMA/SARIMA found that these models struggle when predicting based solely on historical price data. Therefore, these models may be more suitable for this project, which integrates other data sources. This approach also highlights the need to ensure the data is stationary, to investigate the best (p,d,q) values, and to optimize based on AIC/BIC criteria. A future extension noted was the hybridization of ARIMA with CNNs/LSTMs, which will be explored in this project via an ensemble model. Lastly, the work of [5] compares long-term vs. short-term prediction datasets and claims that ARIMA performs better in the short term and struggles with long-term predictions. As this project possess a short and long-term dataset, this claim can be further examined.

2.2 Deep Learning Models

The work in [6] indicates that LSTMs outperformed other deep learning models and suggests including features such as Twitter sentiment and values from other stocks to further test LSTMs. The LSTM in this project will therefore follow as closely to the implementation in [6] to apply this future direction. Work in [7] suggests that min-max scaling in LSTMs helps to reduce bias, which is already the setup in this project. Future overfitting prevention techniques such as dropout should also be explored further.

Most surveys report LSTMs perform well due to their ability to capture temporal information. While other deep learning models such as Transformers were explored in [8], [9], they were not chosen here as they are still relatively new and their effectiveness requires

further exploration. Their nature also makes them less interpretable than the other models investigated. This report therefore explores a mixture of well-defined approaches of varying computational intensity to provide a clear comparative baseline for evaluating the performance and energy consumption of Transformer models in future work.

2.3 Automated ML Tools

Work in [10] compared various automated tools including AutoGluon, AutoSklearn, and PyCaret. The study found all to be strong candidates for AutoML, with AutoGluon excelling in terms of customisability, automated data preparation, feature engineering, and performance.

According to [11], AutoGluon was designed for non-coding professionals, allowing easy usage once the data is in the correct tabular format. Alternatives such as TPOT were explored in [12], but AutoGluon was chosen due to its diversity and ease of implementation. Demonstrated by [13], [14], AutoGluon provides a robust and comprehensive benchmark to compare the final models in terms of performance and ease of implementation. As AutoGluon is distinct in nature, it will be explored as an alternative with a slightly different setup from the rest of the models explored.

3 Experimental Setup

3.1 Models Selected & Models Excluded

From the literature review explored above and the aim of this project, the following models will be implemented:

- Linear Regression (Regularized)
- ARIMA and ARIMAX
- Univariate and Multivariate LSTMs
- AutoGluon (as an alternative)

3.2 Evaluation Design & Time-Series Cross Validation

From the literature review conducted, the most common and recurring metrics were found to be:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)
- Mean Absolute Scaled Error (MASE)
- Symmetric Mean Absolute Percentage Error (sMAPE)

These metrics were further investigated [15], but it was not deemed necessary to deploy all of these metrics as this seemed excessive and redundant. MAE was chosen as it is the most interpretable, representing the average deviation from the actual to the predicted value. The formula for MAE is given by:

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

RMSE was chosen over its counterpart MSE as it provides results on the same scale as the original data by using the square root, making it easier to interpret than MSE. The formula for RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}$$

MAPE is the percentage version of MAE but can be problematic if there are zero values in the denominator, i.e., if the true value is zero. While this is unlikely to be the case in this project, its counterpart sMAPE was chosen as it removes this division issue and looks at over- and under-forecasting equally. The formula for sMAPE is:

$$\text{sMAPE} = \frac{100\%}{n} \sum_{t=1}^n \frac{|y_t - \hat{y}_t|}{(y_t + \hat{y}_t)/2}$$

Time-Series specific cross validation is also extremely important to ensure a lack of data leakage and therefore, a rolling cross validation was created using sklearn's Time series split. These metrics and cross validation formulae were written in a separate py file for modularity and re-usability and can be easily edited and uploaded into any model notebook for evaluation. CodeCarbon will be implemented in every modeling notebook to compare environmental costs such as emissions and total energy consumed.

4 Methodology

Produced in the first assignment, two datasets were created:

1. A long-term dataset consisting of various Bitcoin (BTC) close prices and close prices of various other stocks/ETFs.
2. A short-term dataset consisting of the same data as above but also include Twitter Sentiment Scores towards BTC using VADER Sentiment Extractor.

Various BTC Close prices were extracted due to conflicting data extraction methods. This project focuses on Close.UTC.00 as the target. Each model needs for the data to be processed in a slightly different manner, with varying optimization methods. These are now explored below.

4.1 Linear Regression

The first model implemented was a simple Linear Regression. However, due to its linear nature, independence of its predictors must be further discussed. As mentioned in the first report, high VIF scores between S&P 500 ETF Trust and Apple and Coinbase indicated multicollinearity. For Linear Regression, it assumes that the target variable is some linear combination of multiple independent predictors. Therefore this multicollinearity must be addressed. Dropping S&P 500 ETF Trust reduced the VIF to an acceptable amount and so three experiments were devised:

1. 'X = Close Price BTC, Y = Sentiment Scores, Close Apple, Close Coinbase'
2. 'X = Close Price BTC, Y = Sentiment Scores, Close SPDR S&P 500 ETF Trust'
3. 'X = Close Price BTC, Y = Sentiment Scores, PCA(Close Apple, Close Coinbase, Close SPDR S&P 500 ETF Trust)'

Principal Components Analysis is used in method three to reduced the dimensionality and correlation between the stock predictors. Adding Lag features also raises correlation concerns and are addressed throughout the implementation. The values were already scaled using a min-max scaler, they must be rescaled for PCA using a z-score scaler to capture the variance needed for PCA.

Performance was measured by comparing Mean Absolute Error, Root Mean Squared Error and Symmetric Mean Absolute Percentage Error. AIC, BIC and R Squared will also be used to determine the best mix of predictors. As mentioned, these functions were saved in a separate .py file and loaded in for evaluation. It was found that temporal only and PCA combined are likely to influence Bitcoin Close value. These results were just from initial experiments, before overfitting was explored and extra engineered features were included. The R squared results claims very high values such as 0.994 for the temporal and PCA models, likely indicating overfitting. Full results can be found in the model notebook.

Relative Strength Index (RSI) and lagged features (BTC Close and RSI) were added to the dataset. VIF was used to determine any further Multicollinearity other than the ones already listed above. While most of the other models can deal with multicollinearity, it is still important to note and to deal with in linear models such as the model here. A Multicollinearity warning was consistent whenever YEAR was present so therefore it was removed for further studies. To ensure a fair comparison with the other models, the a Regularized Linear model was fit on all the same predictors as the other models, with the idea that the regularization should combat some of the multicollinearity.

Optuna was chosen to determine the optimal hyperparameters for this regularized model. It provides an

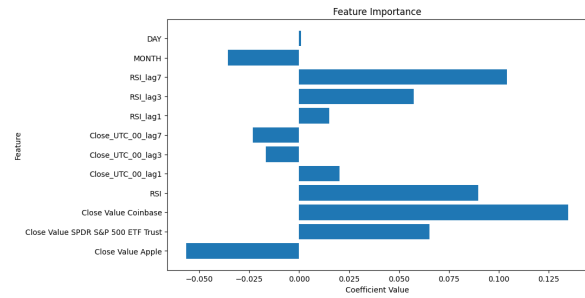


Figure 1: Feature Importance for Dataset 1

efficient way to create a grid over which to determine the best parameters. In this instance, it was also be used to decide whether Ridge, Lasso or something in between would work best in terms of regularization. This optimized 'alpha' and 'L1_wt'. Alpha determines the strength of the penalization for large weights and L1_wt determines whether the regularisation technique acts more like a Lasso Regularization (1) or a Ridge Regularization (0). Alpha was found to be 0.0014 and 0.0015 for Datasets 1 and 2 respectively. L1_wt was found to be 0.0087 and 0.92 for Datasets 1 and 2 respectively. This means the model acts more like a Lasso Regression for Dataset 1 with a very low penalty and a Ridge Regression for Dataset 2. After these parameters were determine the model was refit on an unseen test data and plotted. Feature importance was also extracted and an example is included in Fig.1, highlighting the strong importance of Coinbase and RSI. The model for dataset 1 and 2, as well as the emissions were downloaded for future use.

4.2 ARIMA & ARIMAX

AutoRegressive Integrated Moving Average or ARIMA consists of three major components:

- AR(p): Autoregressive , which is a measure of how much the past influences the present.
- I (d): How many times the data must be differenced to make it stationary (requirement).
- MA (q): Moving Average: how much past erroneous predictions influence the current value.

ARIMA and ARIMAX expects stationary data and according to the literature [2], performing an ADF test will decide whether the data needs to be converted to stationary data. ARIMAX, allows for other features such as Twitter Sentiment to be included. It was seen that for each data split, that differencing to the first or second order is needed to make the data stationary. Autocorrelation Functions (ACF) and Partial Autocorrelation functions (PACF) were also investigated to help indicate the Auto Regressive (AR/Q) and Moving Average components (MA/P) of the datasets.

This gives an indication that differencing is definitely needed and both indicate a potentially low number for PACF. To optimize performance, auto_model

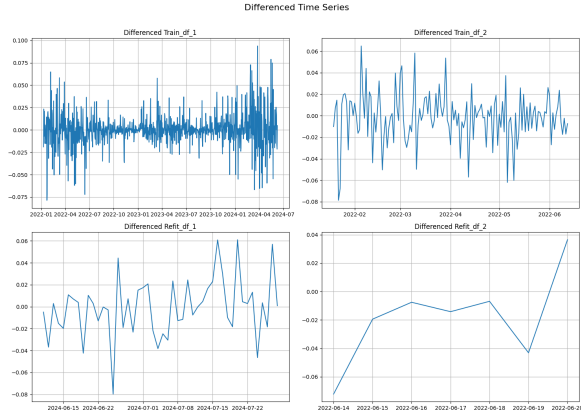


Figure 2: Differencing the Data

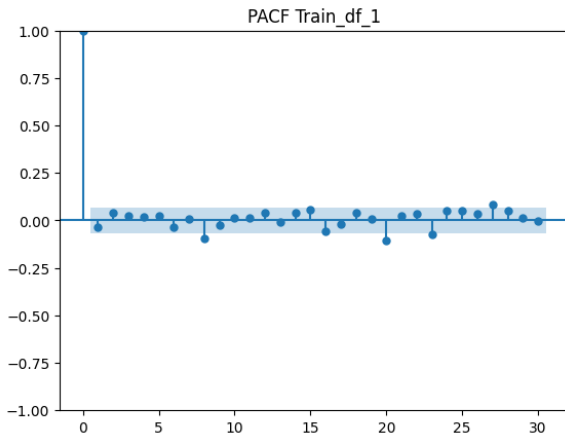


Figure 3: Auto Correlation Function for Dataset 1

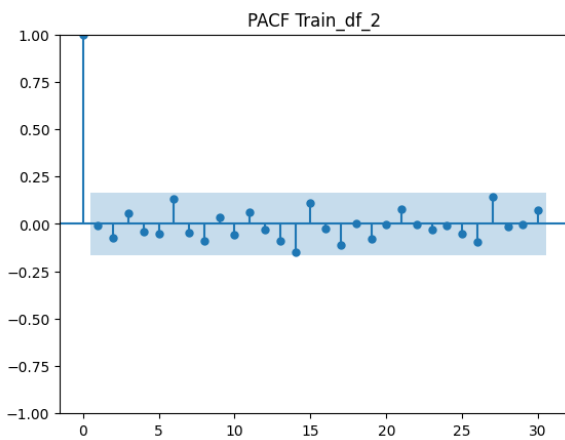


Figure 4: Partial Auto Correlation Function for Dataset 2

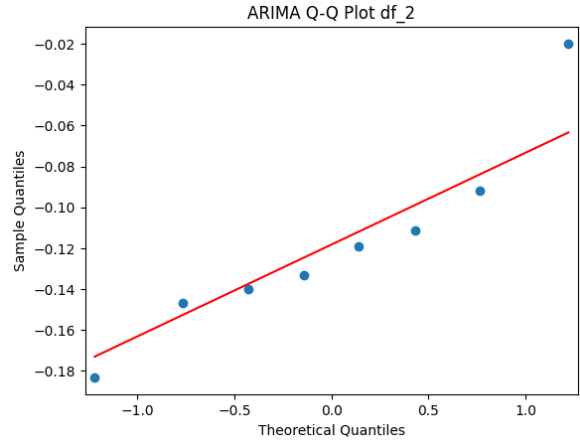


Figure 5: Q-Q Plot for Dataset 2

was used to determine optimal p, d and q values, using AIC to compare performance. This was found to be $(5,2,2)$ for Dataset 1 and $(0,1,0)$ for Dataset 2.

The four models, ARIMA for Dataset 1 and 2 and ARIMAX for Dataset 1 and 2, were then refitted with the optimal (p, d, q) values and the results were visualized in a number of ways. A Quantile-Quantile or Q-Q plot was obtained to investigate the normality of the residuals.

It can be seen in the figure above that the residuals fall fairly close to the straight line, indicating a normal distribution. The p -values from a Ljung-Box Test were also obtained and for the above dataset appear to indicate that the residuals are random. However, this varies from model to model, and therefore autocorrelation and model underfitting cannot be ruled out and a thorough investigation is needed in future directions of this project. This is particularly true with dataset 1, indicating a long-term pattern that needs to be further investigated. All four models were therefore saved, along with their overall emissions.

4.3 LSTM Implementation

Long Short Term Memory models are inherently capable of capturing a temporal structure by retaining memory via a gated architecture. It was decided to test a univariate (Close Prices Only) and a multivariate (i.e. using all predictors) approach to this project. The design of LSTMs differs entirely to the prior models, as they require a window size of history to process and therefore two history functions were created, one for univariate and the other for multivariate. According to [6], LSTM implementations outperformed a variety of other Deep Learning approaches for Bitcoin Price Prediction and recommended testing this model with a multivariate approach, such as including twitter sentiment or other stock values. This will be attempted in this project and therefore the LSTM implementation will be kept as close to the Nair et al. implementation as possible.

The original implementation for the LSTM is well designed in the original paper and described below:

- Step size of 7 (i.e. the history)
- First LSTM layer of 50 nodes (7,50) and a ReLU activation function
- Second LSTM layer of 25 nodes (1,25) and a ReLU activation function
- Fully connected (Dense) layer of 50 nodes.
- Last fully connected (Dense) layer of 1 node - output.
- Use of Adam optimizer - calculates the learning rate.

It is not clear which metric it optimizes on and MAE was decided for this project. The following adjustments will be made to the model:

- Dropout layers will be added to prevent overfitting - this rate will be tuned.
- Other Hyperparameters to be tuned include, choice of epoch c and learning rate.

There were therefore four model, two univariate (dataset 1 and dataset 2) and two multivariate (dataset 1 and dataset 2). Similarly to Linear Regression, Optuna was used to create a hyperparameter grid to search over. Due to the size and complexity of LSTM models, the number of trials had to be greatly reduced (down to 2) in order to run the notebook with the session crashing. It is likely that optimal hyperparameters could be further refined on a more powerful device.

The optimal parameters can be seen in table below:

Model Type	Dropout Rate	Learning Rate	Epochs
Uni(df_1)	0.148	0.0057	63
Uni(df_2)	0.115	0.0026	58
Multi(df_1)	0.369	0.0033	51
Multi(df_2)	0.198	0.0010	75

Table 1: Optimal Hyperparameters for LSTM Models on df_1 and df_2

Once these optimal hyperparameters were obtained, the models were refitted on an unseen test set (separate from the final unseen test set used in the ensemble). Predictions of this fit were extracted to visualize an optimistic fit of the model and an example of this can be seen in Figure below. As with the other approaches, all models and emissions were saved for further use and analysis.

4.4 Ensemble: Meta-Learner

As each model varies with respect to the setup and type of input data, ensembling the models poses some difficulty. It was decided that the best way to include and combine each model was to get each model to predict



Figure 6: Multivariate LSTM Refit on Test Set (Dataset 1).

on the unseen test set individually. Then these predictions were combined and averaged to produce an ensemble result. Future work could explore combining and using a weighted average, however a larger unseen test set would likely be needed to train these weights.

4.5 AutoGluon Alternative

As described in the literature review section, AutoGluon was investigated as an Automated ML alternative. Due to its distinctive nature, it was not used for the final ensemble model and was tested merely as an alternative and cross validated on the entire dataset. As AutoGluon requires tabular data, a specific rolling cross validation was written and automatic evaluation metrics were calculated using sklearn. The predictor function was set to best quality and the time limit was set at 4 minutes. For both Datasets, the best model was found to be 'WeightedEnsemble.L2'. While not pursued further in this project, Autogluon provides a streamlined ML process that could immediately be integrated to get an initial idea for the type of model suited for the data given. Though more custom models would later be needed for better interpretability and analysis.

5 Results

Each model was used to predict on a help-out (5%) test set. The results, including the average ensemble are included in the table below. Upon visualization, the predictions from ARIMA and ARIMAX appeared the worst, closely followed by Linear Regression. The visualisations for the top three approaches are shown below:

5.1 Environmental Comparison

The overall emission and energy usage from each notebook were saved for comparison. It should be noted that these figures reflect the entire notebook, i.e. hyperparameter tuning and multiple models (if applicable). It was decided to keep the monitoring this way to get an idea how much the overall implementation would take. The emissions and energy consumed for the three main approaches: LSTM, Linear Regression and ARIMA/ARIMAX are described in Figure below:

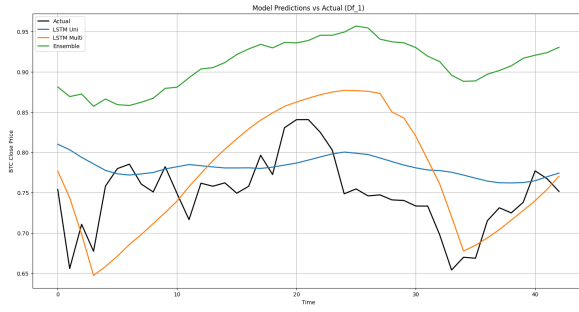


Figure 7: Predictions for Dataset 1

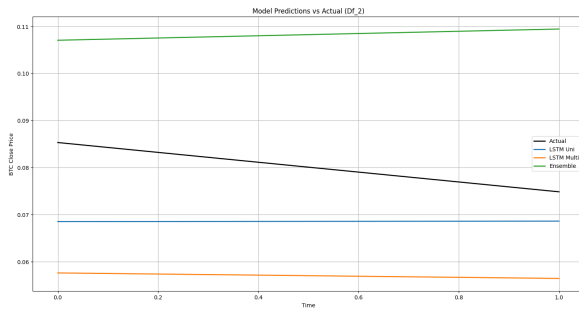


Figure 8: Predictions for Dataset 2

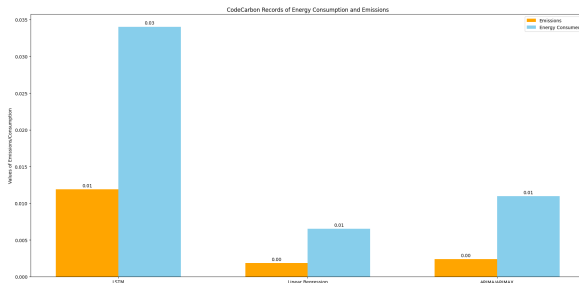


Figure 9: Energy and Emissions Comparison

Model	df_1		
	MAE	RMSE	SMAPE
Linear Regression	0.0652	0.0797	8.67
ARIMA	0.3327	0.3371	36.43
ARIMAX	0.3742	0.3858	39.80
LSTM Uni	0.0433	0.0545	5.75
LSTM Multi	0.0549	0.0665	7.14
Ensemble Model	0.1597	0.1652	19.37

Model	df_2		
	MAE	RMSE	SMAPE
Linear Regression	0.0961	0.0971	200.0
ARIMA	0.1392	0.1393	93.11
ARIMAX	0.0481	0.0492	46.18
LSTM Uni	0.0115	0.0127	15.27
LSTM Multi	0.0231	0.0235	33.44
Ensemble Model	0.0282	0.0289	30.09

Table 2: Evaluation Metrics for Models on df_1 and df_2 Datasets

It can be seen that LSTM incurs a high energy and computational cost, though this is included the Optuna optimization that contributed to the extreme runtime.

6 Conclusion & Future Directions

The results indicate that the LSTM, especially the univariate LSTM outperform the other models in terms of performance. The extremely high SMAPE score for Linear Regression for dataset 1, indicate its low performance and perhaps inability to capture the nuances of this task. However, further work should be explored with the initial Linear Regression models, such as the PCA combination of the stocks to reduce multicollinearity. While this was not included in the final model, to ensure fair comparison to the other models, this final lack in performance could be caused by underlying correlation and similar issues. A future experiment would be to apply the regularization to the various combinations of reduced datasets to improve performance. The Ensemble model did not outperform the LSTM, likely due to the varying performance of the combined models. Moving forward a weighted average ensemble could be explored, or ensembles with respect to model types could further be tested, i.e. the LSTM-based models together as an ensemble etc. However, while best performing, the LSTM-based still requires rigorous further evaluation and investigation. One particular area would be in computational efficiency. The LSTM approach consumed over three times the amount of energy and also had a higher emission rate and overall emissions. This was largely due to the lengthy optimization process, that was restricted already due to computational resources. Therefore, this project introduced a framework for comparing models, both performance wise and with respect to the environmental costs. The results indicate that LSTM far outperformed the other models, but there needs to be further work

into performance-energy balance and improvements to its efficiency to be considered for large scale deployment.

7 AI Usage Statement

As mentioned in this report, the rise of Large Language models and their usage within education is becoming increasingly prominent. Used ethically, they provide an excellent learning resource and therefore the use of LLMs is declared within this report, including types of prompts and models used for transparency.

Models & Prompts Used

ChatGPT 4o (OpenAI)^a

Used for grammar correction and improvement, resource generator, and Python/L^AT_EX code debugging.

Examples:

- "Is Sentence 'X' grammatically correct?"
- "Provide practical resources for LSTM implementation."
- "Explain why this code 'X' raises a Value Error 'Y'?"

Gemini (Google)^b

Explain Error Built into Google Colab.

Example:

- "Explain error: 'ValueError: Y'?"

Claude Sonnet 3.7 (Anthropic) & DeepSeek R1 Model^c

Code Error Correction and Explanation.

Example:

- "Explain error: 'ValueError: Y'?"

^a<https://chat.openai.com>

^b<https://ai.google.dev>

^c<https://claude.ai> https://chat.deepseek.com/sign_in

All information provided by LLMs, such as code suggestions, were thoroughly reviewed and LLMs were always asked to provide resources for their answers. Therefore any information included is accurate to the best understanding of the author.

8 References

[1] N. Uras, L. Marchesi, M. Marchesi, and R. Tonelli, "Forecasting Bitcoin closing price series using linear regression and neural networks models," *PeerJ Computer Science*, vol. 6, p. e279, 2020.

[2] J. Zhu, "Bitcoin Price Prediction: ARIMA & SARIMA vs Linear Regression," *Advances in Economics Management and Political Sciences*, vol. 61, no. 1, pp. 47–54, Dec. 2023, doi: <https://doi.org/10.54254/2754-1169/61/20230776>.

[3] A. Azari, "Bitcoin price prediction: An ARIMA approach," *arXiv preprint arXiv:1904.05315*, 2019.

[4] J. Rebane, I. Karlsson, S. Denic, and P. Papapetrou, "Seq2Seq RNNs and ARIMA models for cryptocurrency prediction: A comparative study," in *FinTech-KDD*, 2018.

[5] A. Alkamali, "Bitcoin Short-term Price Prediction Using Time Series Analysis," Master's thesis, Rochester Institute of Technology, 2024. Available: <https://repository.rit.edu/theses/11695>.

[6] M. Nair, M. I. Marie, and L. A. Abd-Elmegid, "Prediction of Cryptocurrency Price using Time Series Data and Deep Learning Algorithms," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 8, 2023.

[7] J. Fleischer, et al., "Time series analysis of blockchain-based cryptocurrency price changes," *arXiv preprint arXiv:2202.13874*, 2022.

[8] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:10148-10167, 2024.

[9] J. Hao and Q. Sun, "An Adaptive Learning Time Series Forecasting Model Based on Decoder Framework," *Mathematics*, vol. 13, no. 3, p. 490, 2025, doi: <https://doi.org/10.3390/math13030490>.

[10] A. T. Sreedhara and J. Vanschoren, "Can time series forecasting be automated? A benchmark and analysis," *arXiv preprint arXiv:2407.16445*, 2024.

[11] O. Shchur et al., "AutoGluon-TimeSeries: AutoML for Probabilistic Time Series Forecasting," in *AutoML Conference 2023 (ABCD Track)*. Available: <https://openreview.net/forum?id=XHIY3cQ8Tew>.

[12] L. Parmentier, O. Nicol, L. Jourdan, and M.-E. Kessaci, "TPOT-SH: A Faster Optimization Algorithm to Solve the AutoML Problem on Large Datasets," *Proceedings of ICTAI*, pp. 471–478, Nov. 2019, doi: <https://doi.org/10.1109/ictai.2019.00072>.

[13] F. Conrad, M. Mälzer, F. Lange, H. Wiemer, and S. Ihlenfeldt, "AutoML Applied to Time Series Analysis Tasks in Production Engineering," *Procedia Computer Science*, vol. 232, pp. 849–860, 2024.

[14] G. Wastergaard, U. Erden, O. Abdallah Mateo, S. Musah Lampo, T. Çetin Akıncı, and O. Topsakal, "Time Series Forecasting Utilizing Automated Machine Learning (AutoML): A Comparative Analysis Study on Diverse Datasets," *Information*, vol. 15, no. 1, p. 39, Jan. 2024, doi: <https://doi.org/10.3390/info15010039>.

[15] E. Suleman, "Understanding Evaluation Metrics for Time Series Forecasting," Medium, Jul. 04, 2024. <https://eshban9492.medium.com/understanding-evaluation-metrics-for-time-series> (accessed Apr. 16, 2025).